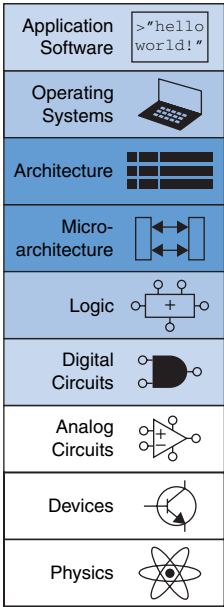


8.1 引言

计算机系统的性能既取决于处理器微架构，也取决于存储系统。第7章假设了一个理想的存储系统，可以在单个时钟周期内访问。然而，这仅适用于非常小的存储器——或者非常慢的处理器！早期处理器相对较慢，因此存储器能够跟上其速度。但处理器速度的增长速度快于存储器速度。DRAM存储器目前比处理器慢10到100倍。处理器与DRAM存储器速度之间日益扩大的差距，要求设计越来越巧妙的存储系统，以尽可能接近与处理器速度相当的存储器。本章前半部分将探讨存储系统，并权衡速度、容量和成本之间的关系。

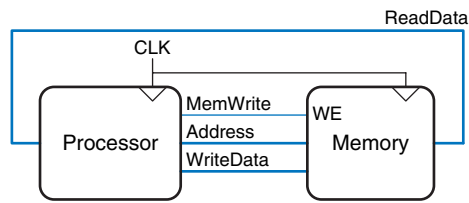
处理器通过*memory interface*与存储系统通信。图8.1展示了我们的多周期RISC-V处理器中使用的简单存储接口。处理器通过*Address*总线向存储系统发送地址。对于读操作，*MemWrite*为0，存储系统通过*ReadData*总线返回数据。对于写操作，*MemWrite*为1，处理器通过*WriteData*总线向存储系统发送数据。

内存系统设计中的主要问题可以大致用图书馆中书籍的比喻来解释。图书馆的书架上存放着许多书。如果你正在写一篇关于梦境意义的学期论文，你可能会去图书馆¹，从书架上取下弗洛伊德*The Interpretation of Dreams*的书，带到你的小隔间里。翻阅之后，你可能会把它放回去，再拿出荣格*The Psychology of the Unconscious*的书。接着，你可能会再次回去查找*Interpretation of Dreams*的另一段引文，然后又去书架取弗洛伊德*The Ego and the Id*的书。很快，



¹ We realize that library usage is plummeting among college students because of the Internet. But we also believe that libraries contain vast troves of hard-won human knowledge that are not electronically available. We hope that Web searching does not completely displace the art of library research.

图8.1 内存接口



你会厌倦从工位走到书架。如果聪明的话，你会把书留在工位而不是来回搬运，以此节省时间。此外，当你抽出弗洛伊德的一本书时，还可以从同一书架上取下他的其他几本书。

这一比喻强调了第6.2.1节中引入的原则，即让常见情况更快。通过将最近使用过或未来可能用到的书籍放在办公桌旁，可以减少前往书库的耗时次数。具体而言，这里运用了*temporal*和*spatial locality*的原则。时间局部性意味着，如果你最近使用过某本书，你很可能很快会再次使用它。空间局部性则意味着，当你使用某一本书时，你很可能对同一书架上的其他书籍也感兴趣。

该库本身通过利用这些局部性原理使常见情况变得快速。图书馆既没有书架空间，也没有预算来容纳世界上所有的书籍。相反，它将一些较少使用的书籍存放在地下室的深层储藏室中。此外，它可能与附近的图书馆有馆际互借协议，从而能够提供比其实际藏书更多的书籍。

总之，通过存储层次结构，您既能拥有庞大的藏书量，又能快速获取最常用的书籍。最常用的书籍放在您的隔间里，更大的藏书量在书架上，而提前通知后，还能从地下室和其他图书馆获取更庞大的藏书。类似地，内存系统也采用存储层次结构，以便快速访问最常用的数据，同时仍具备存储大量数据的能力。



用于构建此层次结构的内存子系统已在第5.5节中介绍。计算机内存主要由动态随机存取存储器（DRAM）和静态随机存取存储器（SRAM）构成。理想情况下，计算机内存系统应具备快速、大容量且廉价的特点。然而在实践中，单一内存只能满足这三个属性中的两个：它要么速度慢，要么容量小，要么价格昂贵。但计算机系统可以通过结合快速、小容量、廉价的内存与慢速、大容量、廉价的内存来接近理想状态。快速内存存储最常用的数据和指令，因此平均而言，内存系统显得快速。大容量内存存储其余的数据和指令，因此整体容量很大。两种廉价内存的组合远低于单一大型快速内存的成本。这些原理可扩展至使用整个容量递增、速度递减的内存层次结构。

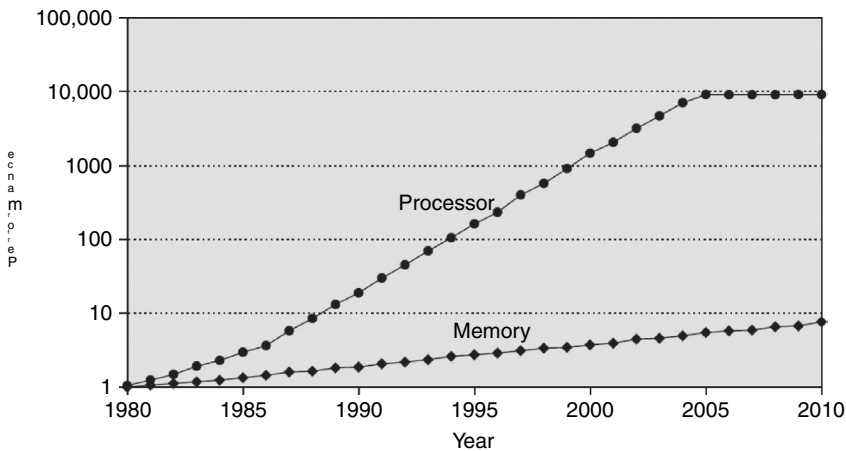


图8.2 处理器与内存性能的分化。经Hennessy和Patterson许可改编，*Computer Architecture: A*，第5版，Morgan Kaufmann，2011年

请记住，速度由延迟和吞吐量共同决定。内存延迟是指访问第一个字节信息所需的时间。吞吐量是每秒能够传输的字节数。许多内存具有高吞吐量但延迟较长。

计算机内存通常由DRAM芯片构成。2021年，一台典型PC的 *main memory* 包含8至32 GiB的DRAM，DRAM价格约为每吉比字节（GiB）3美元。过去三十年间，DRAM价格以每年15%至25%的幅度下降，内存容量也以相同速率增长，因此PC中内存的总成本大致保持恒定。遗憾的是，DRAM速度每年仅提升约7%，而处理器性能却以每年25%至50%的速率提升，如图8.2所示。该图以1980年的速度为基准，展示了内存（DRAM）与处理器的速度变化。大约在1980年，处理器与内存速度相当。然而，此后性能差距逐渐拉大，内存速度严重滞后。²

在20世纪70年代和80年代初期，DRAM能够跟上处理器的速度，但现在它已经慢得令人遗憾。DRAM的访问时间比处理器周期时间长一到两个数量级（几十纳秒，而处理器周期时间不到一纳秒）。DRAM的吞吐量不错，大约为30 GB/s。

为应对这一趋势，计算机将最常用的指令和数据存储在一种速度更快但容量较小的存储器中，称为 *cache*。缓存通常由SRAM构成，与处理器位于同一芯片上。由于SRAM天生比DRAM更快，且片上存储器消除了往返于独立芯片所造成的长时间延迟，因此缓存速度可与处理器速度相媲美。

² Although recent single-processor performance has remained approximately constant, as shown in Figure 8.2 for the years 2005 to 2010, the increase in multicore systems (not depicted on the graph) only worsens the gap between processor and memory performance.

2021年，片上SRAM的成本约为100美元/GiB，但由于缓存容量相对较小（从几千字节到几兆字节），总体成本较低。缓存可以同时存储指令和数据，但我们将统称其内容为“数据”。SRAM的延迟范围从16 KiB缓存的零点几纳秒到4 MiB缓存的几纳秒，吞吐量可达数百GB/s。

如果处理器请求的数据在缓存中可用，则会快速返回。这称为缓存 *hit*。否则，处理器会从主存（DRAM）中检索数据。这称为缓存 *miss*。如果缓存大部分时间都能命中，那么处理器很少需要等待缓慢的主存，平均访问时间就会很低。

存储器层次结构中的第三级是硬盘。就像图书馆用地下室存放书架上放不下的书籍一样，计算机系统用硬盘来存储主存中放不下的数据。2021年，采用磁存储技术的硬盘驱动器（HDD）成本低于0.03美元/GB，访问时间约为5至10毫秒。大文件的吞吐量约为100 MB/s，而对于小文件（4 KiB）的随机访问，吞吐量可降至1 MB/s。硬盘成本每年下降60%，但访问时间几乎没有改善。采用闪存技术的固态硬盘（SSD）正日益成为HDD的常见替代品。SSD在利基市场已使用超过二十年，并于2007年进入主流市场。SSD克服了HDD的一些机械故障，但成本是HDD的3到4倍，约为0.10美元/GB。自SSD上市以来，其与HDD之间的价格差距不断缩小，SSD相对于HDD的普及度也随之提高。SSD的访问时间小于0.1毫秒。大文件的吞吐量可达500至3,000 MB/s，而对于4 KiB的小文件，吞吐量可降至50至250 MB/s。

硬盘提供的容量比主存实际存在的容量更大，因此被称为 *virtual memory*。就像地下室里的书籍一样，虚拟内存中的数据访问需要很长时间。主存，也称为 *physical memory*，保存了虚拟内存的一个子集。因此，主存可以被视为硬盘中最常用数据的缓存。

图8.3总结了本章其余部分讨论的计算机系统的内存层次结构。处理器首先在一个通常位于同一芯片上的小型但快速的缓存中寻找数据。如果

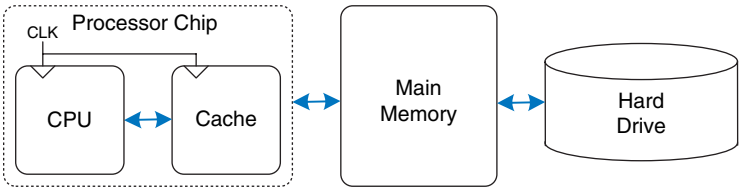


图 8.3 典型的内存层次结构

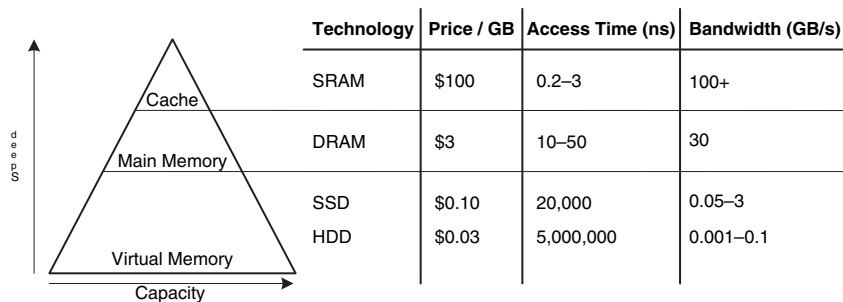


图8.4 内存层次结构组件，2021年的典型特征

缓存中无数据时，处理器会继续在主存中查找。若主存中也无数据，处理器则从容量大但速度慢的硬盘虚拟内存中获取数据。图8.4展示了内存层次结构中容量与速度的权衡，并列举了2021年技术下的典型成本、访问时间及带宽。随着访问时间缩短，速度随之提升。

第8.2节介绍了内存系统性能分析。第 8.3节探讨了几种缓存组织方式，第8.4节深入研究了虚拟内存系统。

8.2 内存系统性能分析

设计师（以及计算机购买者）需要量化的方法来衡量内存系统的性能，以评估各种替代方案的成本效益权衡。内存系统的性能指标包括miss、rate或hit rate以及average memory access time。未命中率和命中率的计算方式如下：

$$Miss Rate = \frac{Number\ of\ misses}{Number\ of\ total\ memory\ accesses} = 1 - Hit Rate$$

$$Hit Rate = \frac{Number\ of\ hits}{Number\ of\ total\ memory\ accesses} = 1 - Miss Rate$$

(8.1)

示例 8.1 计算缓存性能

假设一个程序有2,000条数据访问指令（加载或存储），其中1,250条请求的数据值在缓存中找到。其余750条数据值由主存或磁盘存储器提供给处理器。缓存的未命中率和命中率分别是多少？

解：未命中率为 $750/2000 = 0.375 = 37.5\%$ 。命中率为 $1250/2000 = 0.625 = 1 - 0.375 = 62.5\%$ 。

Average memory access time(AMAT) 是处理器每条加载或存储指令平均需要等待内存的时间。在图8.3所示的典型计算机系统中，处理器首先在缓存中查找数据。如果缓存未命中，处理器随后在主存中查找。如果主存也未命中，处理器则访问硬盘上的虚拟内存。因此，AMAT 的计算方式为：

$$AMAT = t_{\text{cache}} + MR_{\text{cache}}(t_{MM} + MR_{MM}t_{VM}) \tag{8.2}$$

其中 t_{cache} 、 t_{MM} 和 t_{VM} 分别是缓存、主存和虚拟内存的访问时间， MR_{cache} 和 MR_{MM} 分别是缓存和主存的缺失率。

示例8.2 计算平均内存访问时间

假设一个计算机系统的存储器层次结构只有两级：缓存和主存。根据表8.1中的访问时间和未命中率，平均存储器访问时间是多少？

解 平均内存访问时间为 $1 + 0.1(100) = 11$ 个周期。

表8.1 访问时间与缺失率

Memory Level	Access Time (Cycles)	Miss Rate
Cache	1	10%
Main Memory	100	0%

示例 8.3 改善访问时间

11个周期的平均内存访问时间意味着处理器每实际使用数据一个周期，就要花费十个周期等待数据。根据表8.1中的访问时间，需要多大的缓存未命中率才能将平均内存访问时间降低到1.5个周期？

解 如果未命中率为 m ，则平均访问时间为 $1 + 100m$ 。将该时间设为1.5并求解 m ，需要缓存未命中率为0.5%。

需要提醒的是，性能提升的效果可能并不总是听起来那么好。例如，将内存系统速度提升十倍，并不一定能让计算机程序的运行速度也提升十倍。如果程序中50%的指令是加载和存储操作，那么内存系统性能提升十倍仅能使程序性能提升1.82倍。这一通用原则被称为Amdahl's



吉恩·阿姆达尔（1922–2015）最著名的是阿姆达尔定律，这是他在1965年提出的一个观察结论。在读研究生期间，他开始利用业余时间设计计算机。这项副业使他在1952年获得了理论物理学博士学位。毕业后他立即加入IBM，后来相继创立了三家公司，其中包括1970年成立的阿姆达尔公司。

*Law*该原则指出，只有当一个子系统对整体性能影响较大时，为其提升性能所付出的努力才是有价值的。

8.3 缓存

缓存保存常用的内存数据。它能容纳的数据字数量称为 $capacity$ 、 C 。由于缓存的容量小于主存，计算机系统设计者必须选择主存中的哪个子集保留在缓存中。

当处理器尝试访问数据时，会首先检查缓存中是否存在该数据。若缓存命中，数据可立即获取；若缓存未命中，处理器则从主存中获取数据并将其存入缓存以备后续使用。为容纳新数据，缓存必须 $replace$ 旧数据。本节通过回答以下问题来探讨缓存设计中的这些议题：（1）缓存中存放哪些数据？（2）如何查找数据？（3）当缓存已满时，替换哪些数据以腾出空间存放新数据？

在阅读后续章节时，请记住回答这些问题的驱动力是大多数应用中数据访问固有的空间和时间局部性。缓存利用空间和时间局部性来预测接下来需要哪些数据。如果程序以随机顺序访问数据，则无法从缓存中受益。

如下文所述，缓存由其容量（ C ）、组数（ S ）、块大小（ b ）、块数（ B ）以及关联度（ N ）来定义。

尽管我们专注于数据缓存加载，但同样的原则也适用于指令缓存的读取。数据缓存存储操作类似，将在第8.3.4节中进一步讨论。

8.3.1 缓存中保存了哪些数据？

理想的缓存能够预判处理器所需的所有数据，并提前从主存中获取，从而实现零缺失率。由于无法完美准确地预测未来，缓存必须根据过去的内存访问模式来猜测哪些数据将被需要。具体而言，缓存利用时间局部性和空间局部性来实现低缺失率。

回想一下，时间局部性意味着如果处理器最近访问过某块数据，那么它很可能很快再次访问该数据。因此，当处理器加载或存储不在缓存中的数据时，该数据会从主存复制到缓存中。随后对该数据的请求将在缓存中命中。

回想一下，空间局部性指的是，当处理器访问某块数据时，它也很可能访问附近内存位置的数据。

Cache: 一个隐藏处，尤指用于隐藏和保存物资或工具的地方。

—梅里亚姆-韦伯斯特在线词典。2021年。www.merriam-webster.com

因此,当缓存从内存中读取一个字时,它可能也会读取几个相邻的字。这一组字被称为 $cache\ block$ 或 $cache\ line$ 。缓存块中的字数 b 称为 $block\ size$ 。容量为 C 的缓存包含 B 个 $=C/b$ 块。

时间局部性和空间局部性的原理已在真实程序中通过实验验证。如果程序中使用了一个变量,该变量很可能被再次使用,从而产生时间局部性。如果使用了数组中的一个元素,同一数组中的其他元素也很可能被使用,从而产生空间局部性。

8.3.2 数据是如何被发现的?

缓存被组织成 $S\ sets$,每个 $S\ sets$ 包含一个或多个数据块。主存中数据的地址与该数据在缓存中位置之间的关系称为 $mapping$ 。每个内存地址精确映射到缓存中的一个组。部分地址位用于确定哪个缓存组包含该数据。如果该组包含多个块,则数据可以保存在该组中的任意一个块中。

缓存根据每组中的块数进行分类。在 $direct\ mapped$ 缓存中,每组恰好包含一个块,因此缓存有 $S = B$ 组。因此,特定的主存地址映射到缓存中的一个唯一块。在 $N\text{-way}\ set\ associative$ 缓存中,每组包含 N 个块。地址仍然映射到唯一的组,共有 $S = B/N$ 组。但来自该地址的数据可以放入该组中的任意 N 个块。 $fully\ associative$ 缓存只有 $S = 1$ 组。数据可以放入该组中的任意 B 个块。因此,全相联缓存是 B 路组相联缓存的另一种称呼。

为了说明这些缓存组织,我们将考虑一个具有32位地址和32位字的RISC-V内存系统。内存是字节可寻址的,每个字为四个字节,因此内存由 2^{30} 个按字边界对齐的字组成。为简单起见,我们分析容量为八个字(C)的缓存。我们从一字块大小(b)开始,随后推广到更大的块。

直接映射缓存

一个 $direct\ mapped$ 缓存中每个组只有一个块,因此它被组织成 $S = B$ 个组。要理解内存地址到缓存块的映射,可以将主存想象成被映射为 b 字大小的块,就像缓存一样。主存中块0的地址映射到缓存的组0。主存中块1的地址映射到缓存的组1,依此类推,直到主存中块 $B-1$ 的地址映射到缓存的块 $B-1$ 。缓存中没有更多的块了,因此映射会循环,使得主存中块 B 映射到缓存的块0。

该映射关系如图8.5所示,针对一个容量为八个字、块大小为单个字的直接映射缓存。该缓存

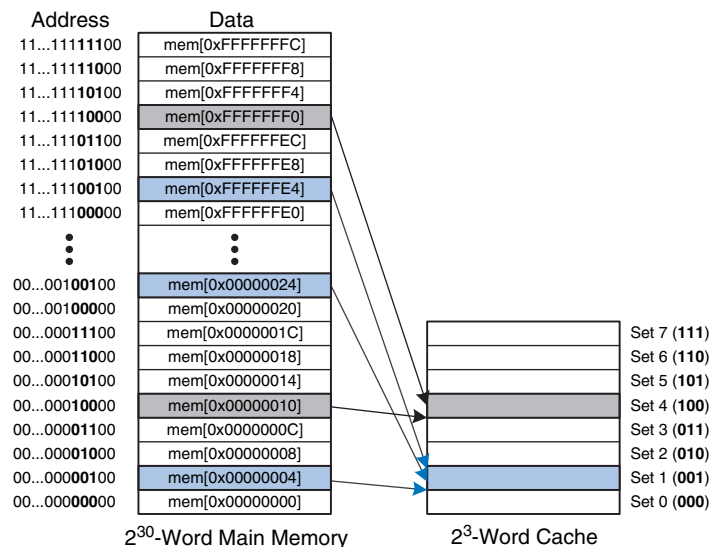


图8.5 主存到直接映射缓存的映射

有八组，每组包含一个单字块。地址的最低两位始终为00，因为它们是字对齐的。接下来的 $\log_2 8 = 3$ 位指示内存地址映射到的组。因此，地址0x00000004、0x00000024、……、0xFFFFF0E4处的数据都映射到组1，如蓝色所示。同样，地址0x00000010、……、0xFFFFF0F0处的数据都映射到组4，依此类推。每个主存地址恰好映射到缓存中的一组。

示例8.4 缓存字段

图8.5中地址0x00000014处的字映射到哪个缓存集？请说出另一个映射到同一集的地址。

解决方案 地址的最低两位为00，因为该地址是字对齐的。接下来的三位是101，因此该字映射到组5。地址为0x34、0x54、0x74、…、0xFFFFF0F4的字都映射到同一组。

由于多个地址映射到同一个集合，缓存还必须跟踪每个集合中实际包含的数据的地址。地址的最低有效位指定了哪个集合保存该数据。其余的最高有效位称为tag，用于指示该集合中保存的是众多可能地址中的哪一个。

在我们之前的示例中，32位地址的最低两位被称为byte offset，因为它们指示字内的字节。接下来的三位被称为set bits，因为它们指示地址映射到的组。（通常，组位的数量是 $\log_2 S$ 。）剩余的27个标签位指示内存地址的

图8.6 映射到图8.5中缓存时，地址0xFFFFFE4的缓存字段



存储在给定缓存集中的数据。图8.6显示了地址0xFFFFFE4的缓存字段。它映射到集合1，其标签全为1。

示例 8.5 缓存字段

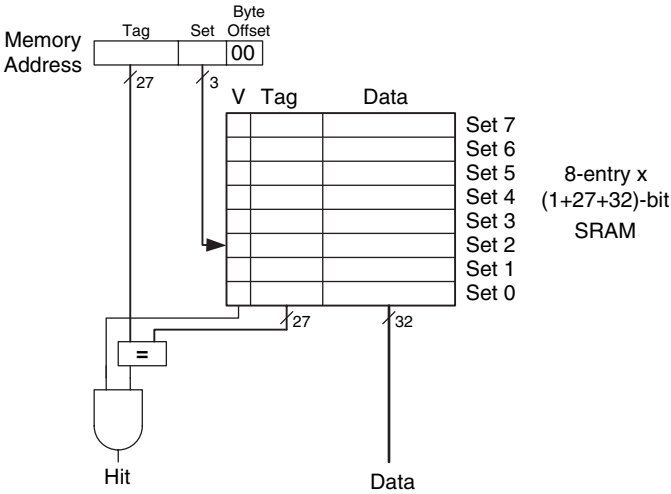
对于具有1024 (2^{10}) 个集合且块大小为一个字的直接映射缓存，求其组索引位和标记位的数量。地址大小为32位。

解决方案 一个包含 2^{10} 组的高速缓存需要 $\log_2(2^{10}) = 10$ 个组位。地址的最低两位是字节偏移量，剩余的 $32 - 10 - 2 = 20$ 位构成标签。

有时，例如在计算机首次启动时，缓存组中不包含任何数据。缓存为每个组使用一个*valid bit*来指示该组是否包含有意义的数据。如果有效位为0，则内容无意义。

图8.7展示了图8.5中直接映射缓存的硬件结构。该缓存由8个条目的SRAM构成。每个条目（或称组）包含一行数据：32位数据、27位标签和1位有效位。缓存通过32位地址进行访问。最低两位（字节偏移位）在字访问时被忽略。接下来的三位（组位）指定缓存中的条目或组。加载指令从缓存中读取指定条目并检查

图8.7 直接映射缓存（8组）



标签位和有效位。如果标签与请求地址的最高27位匹配且有效位为1，则缓存命中，数据返回给处理器。否则，缓存未命中，内存系统必须从主存中获取数据。

示例8.6 直接映射缓存的临时局部性

循环是应用程序中时间和空间局部性的常见来源。使用图8.7中的八条目缓存，展示在RISC-V汇编代码中执行以下愚蠢循环后缓存的内容。假设缓存初始为空。缺失率是多少？

```
addi s0, zero, 5  addi s1, zero, 0  LO
OP: beq s0, zero, DONE  lw s2, 4(s
1) lw s3, 12(s1) lw s4, 8(s1) addi s
0, s0, -1 j LOOP
```

完成：

解决方案 该程序包含一个循环，重复执行五次。每次迭代涉及三次内存访问（加载），总共产生15次内存访问。第一次执行循环时，缓存为空，数据必须从主存地址0x4、0xC和0x8分别加载到缓存组1、3和2中。然而，在接下来的四次循环执行中，数据已在缓存中找到。图8.8显示了最后一次请求内存地址0x4时缓存的内容。所有标签均为0，因为地址的高27位为0。缺失率为 $3/15 = 20\%$ 。

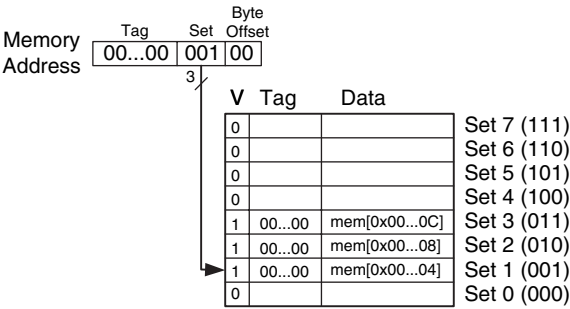


图8.8 直接映射缓存内容

当两个最近访问的地址映射到同一个缓存块时，会发生 *conflict*，并且最近访问的地址 *evicts* 会从该块中替换掉前一个地址。直接映射缓存存在每个

设置，因此映射到同一集合的两个地址总是会导致冲突。示例8.7说明了冲突情况。

示例8.7 缓存块冲突

当以下循环在来自图8.7的八字直接映射缓存上执行时，未命中率是多少？假设缓存初始为空。

```
addi s0, zero, 5  addi s1, zero, 0  LO
OP: beq s0, zero, DONE  lw s2, 0x4(
s1) lw s4, 0x24(s1)  addi s0, s0, -1
j LOOP
```

完成：

解决方案 内存地址 0x4 和 0x24 都映射到 组 1。在循环的初始执行期间，地址 0x4 的数据被加载到缓存的组 1 中。然后，地址 0x24 的数据被加载到组 1，将地址 0x4 的数据逐出。在循环的第二次执行时，模式重复，缓存必须重新获取地址 0x4 的数据，逐出地址 0x24 的数据。这两个地址冲突，缺失率为 100%。

多路组相联缓存

N-way set associative 缓存通过在每个集合中提供 *N* 个块来减少冲突，映射到该集合的数据可以在这些块中找到。每个内存地址仍然映射到特定的集合，但它可以映射到 *N* 个块中的任意一个。

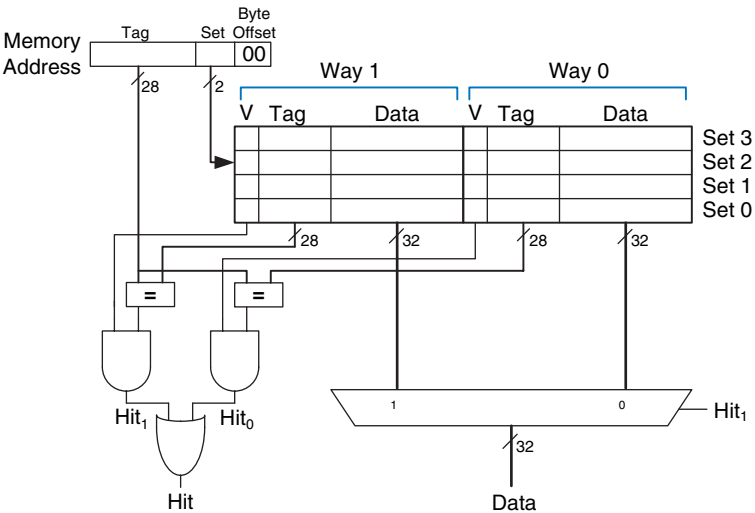


图8.9 两路组相联缓存

在集合中。因此，直接映射缓存是单路组相联缓存的另一种称呼。N也被称为缓存的*degree of associativity*。

图8.9展示了一个采用C=8字、N=2路组相联缓存的硬件结构。该缓存现在仅有S=4个组而非8个，因此仅使用log₂4=2个组选择位（而非3位）来选择组。标签位从27位增加到28位。每个组包含两个*ways*（即相联度）。每一路包含一个数据块以及有效位和标签位。缓存从选定组的两路中读取数据块，并检查标签和有效位以判断是否命中。若其中一路命中，多路选择器将从该路选择数据。

组相联缓存通常比相同容量的直接映射缓存具有更低的缺失率，因为其冲突更少。然而，由于输出多路选择器和额外比较器的存在，组相联缓存通常速度较慢且构建成本较高。此外，当两路均满时，还涉及替换哪一路的问题；这一点将在第8.3.3节进一步讨论。大多数商用系统采用组相联缓存。

示例8.8 组相联缓存缺失率

重复示例8.7，使用图8.9中的八字两路组相联缓存。

解决方案 两次内存访问（地址0x4和0x24）都映射到组1。然而，该缓存有两个路，因此可以容纳来自两个地址的数据。在第一次循环迭代中，空缓存对两个地址均未命中，并将两个数据字加载到组1的两个路中，如图8.10所示。在接下来的四次迭代中，缓存命中。因此，未命中率为2/10 = 20%。回顾一下，示例8.7中相同大小的直接映射缓存的未命中率为100%。

Way 1			Way 0			
V	Tag	Data	V	Tag	Data	
0			0			Set 3
0			0			Set 2
1	00...00	mem[0x00...24]	1	00...10	mem[0x00...04]	Set 1
0			0			Set 0

图8.10 两路组相联缓存内容

全相联缓存

一个*fully associative*缓存包含一个具有B路的单一集合，其中B是块的数量。一个内存地址可以映射到这些路中的任意一个块。全相联缓存是只有一个集合的B路组相联缓存的另一种称呼。

图8.11展示了一个具有八个块的全相联缓存的SRAM阵列。当收到数据请求时，必须进行八次标签比较（未显示），因为数据可能位于任意一个块中。类似地，一个8:1

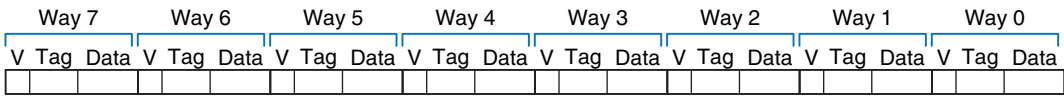


图8.11 八块全相联缓存

多路选择器在命中发生时选择正确的数据。全相联缓存对于给定的缓存容量往往具有最少的冲突缺失，但需要更多硬件来进行额外的标签比较。由于需要大量比较器，它们最适合相对较小的缓存。

区块大小

之前的示例只能利用时间局部性，因为块大小为一个字。为了利用空间局部性，缓存使用更大的块来保存多个连续的字。

块大小大于1的优势在于，当发生未命中并将字取入缓存时，块中的相邻字也会被一并取入。因此，由于空间局部性，后续访问更有可能命中。然而，较大的块大小意味着固定大小的缓存将拥有更少的块。这可能导致更多冲突，从而增加未命中率。此外，未命中后取回缺失缓存块所需的时间更长，因为需要从主存中取回多个数据字。将缺失块加载到缓存所需的时间称为 $miss\ penalty$ 。如果块中的相邻字后续未被访问，则取回它们的努力就白费了。尽管如此，大多数实际程序仍能从较大的块大小中受益。

图8.12展示了一个采用 $C=8$ 字直接映射缓存且块大小为 $b=4$ 字的硬件结构。该缓存现在仅有 $B=C/b=2$ 个块。直接映射缓存的每个组中只有一个块，因此该缓存

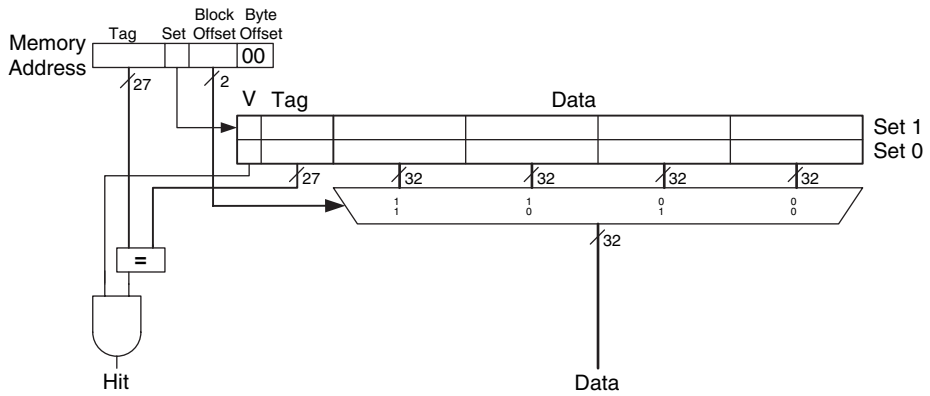


图8.12 直接映射缓存，具有两组和四字块大小

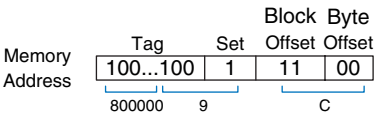


图8.13 映射到图8.12缓存时地址0x8000009C的缓存字段

被组织为两组。因此，仅使用 $\log_2 2 = 1$ 位来选择组。现在需要一个多路选择器来选择块内的字。该多路选择器由地址的 $\log_2 4 = 2$ *block offset bits*控制。最高有效的27位地址构成标签。整个块只需要一个标签，因为块中的字位于连续的地址上。

图8.13展示了地址0x8000009C映射到图8.12中直接映射缓存时的缓存字段。字访问的字节偏移位始终为0。接下来的 $\log_2 b = 2$ 个块偏移位指示块内的字，再下一位指示组。剩余的27位是标签。因此，字0x8000009C映射到缓存中的组1、字3。利用更大块大小来开发空间局部性的原则同样适用于关联缓存。

示例 8.9 直接映射缓存的空间局部性

重复示例8.6，使用八字直接映射缓存，块大小为四字。

解 图8.14展示了第一次内存访问后的缓存内容。在第一次循环迭代中，访问内存地址0x4时发生缓存未命中。此次访问将地址0x0至0xC的数据加载到缓存块中。后续所有访问（如地址0xC所示）均在缓存中命中。因此，未命中率为 $1/15 = 6.67\%$ 。

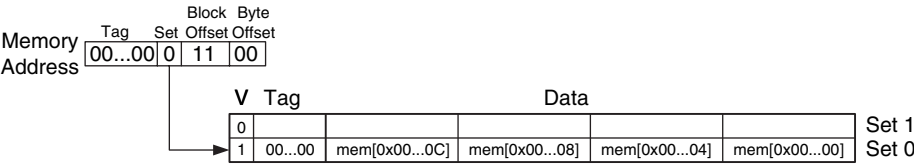


图8.14 块大小为四个字的缓存内容

综合运用
缓存被组织为二维数组。行称为*sets*，列称为*ways*。数组中的每个条目

表8.2 缓存组织

Organization	Number of Ways (<i>N</i>)	Number of Sets (<i>S</i>)
Direct Mapped	1	<i>B</i>
Set Associative	$1 < N < B$	<i>B</i> / <i>N</i>
Fully Associative	<i>B</i>	1

由数据块及其相关的有效位和标签位组成。缓存的特征在于

- 容量 *C* 块大小 *b*（以及块数量， $B = C/b$ ）▶ 每
- 组中的块数量（*N*）

表8.2总结了各种缓存组织方式。内存中的每个地址仅映射到一个组，但可以存储在任何路中。

缓存容量、相联度、组大小和块大小通常为2的幂次。这使得缓存字段（标记、组和块偏移位）成为地址位的子集。

提高关联度 *N* 通常会降低由冲突引起的未命中率。但更高的关联度需要更多的标签比较器。增加块大小 *b* 可以利用空间局部性来降低未命中率。然而，它会减少固定大小缓存中的组数，因此可能导致更多冲突。同时也会增加未命中惩罚。

8.3.3 哪些数据被替换？

在直接映射缓存中，每个地址映射到唯一的块和组。当需要加载新数据而该组已满时，该组中的块会被新数据替换。在组相联和全相联缓存中，当缓存组已满时，缓存必须选择要驱逐的块。时间局部性原理表明，最佳选择是驱逐最近最少使用的块，因为它最不可能很快被再次使用。因此，大多数相联缓存采用*least recently used*（LRU）替换策略。

在二路组相联缓存中，*use bit*、*U*表示组内哪一路最近最少使用。每次使用其中一路时，*U*会被调整以指示另一路。对于超过两路的组相联缓存，追踪最近最少使用的路变得复杂。为简化问题，通常将路分为两组，*U*指示哪一组*group*路最近最少使用。替换时，新块会随机替换

在最近最少使用组内的块。这种策略称为*pseudo-LRU*，在实践中足够好。

示例 8.10 LRU 替换

展示执行以下代码后，一个八字双向组相联缓存的内容。假设采用LRU替换策略，块大小为单字，且缓存初始为空。

```
addi t0, zero, 0
lw s1, 0(x4(t0))
lw s2, 0x24(t0)
w s3, 0x54(t0)
```

解决方案 前两条指令将内存地址0x4和0x24中的数据加载到缓存组1中，如图8.15(a)所示。*U* = 0表示路0中的数据是最近最少使用的。接下来对地址0x54的内存访问同样映射到组1，并替换路0中最近最少使用的数据，如图8.15(b)所示。使用位*U*被设置为1，以指示路1中的数据是最近最少使用的。

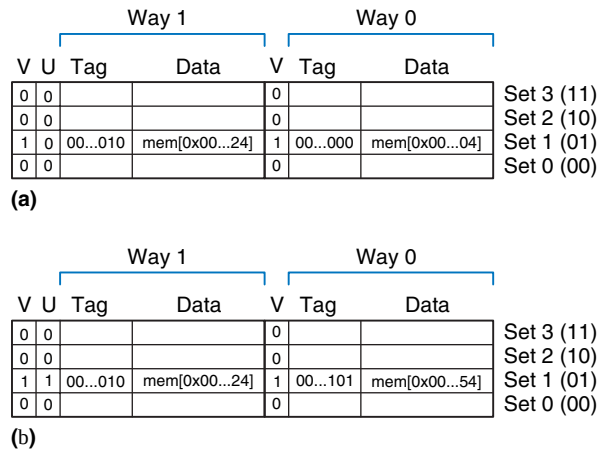
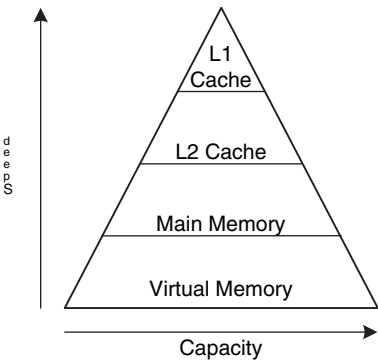


图8.15 使用LRU替换策略的两路组相联缓存

8.3.4 高级缓存设计*

现代系统使用多级缓存来减少内存访问时间。本节探讨两级缓存系统的性能，并研究块大小、关联度和缓存容量如何影响未命中率。本节还描述了缓存如何通过使用写直达或写回策略来处理存储操作（即写入操作）。

图8.16 具有两级缓存的存储器层次结构



多级缓存

大容量缓存是有益的，因为它们更有可能保存感兴趣的数据，从而降低未命中率。然而，大容量缓存往往比小容量缓存速度更慢。现代系统通常至少使用两级缓存，如图8.16所示。第一级（L1）缓存足够小，可以提供一到两个周期的访问时间。第二级（L2）缓存也由SRAM构建，但比L1缓存更大，因此也更慢。处理器首先在L1缓存中查找数据。如果L1缓存未命中，处理器会在L2缓存中查找。如果L2缓存也未命中，处理器则从主内存中获取数据。由于访问主内存速度非常慢，许多现代系统在内存层次结构中增加了更多级别的缓存。

示例 8.11 具有 L2 缓存 的系统

使用图8.16的系统，其中L1缓存、L2缓存和主存的访问时间分别为1、10和100个周期。假设L1和L2缓存的缺失率分别为5%和20%。具体来说，在5%未命中L1缓存的访问中，有20%也未命中L2缓存。平均内存访问时间（AMAT）是多少？

解决方案 每次内存访问都会检查L1缓存。当L1缓存未命中时（5%的概率），处理器会检查L2缓存。当L2缓存未命中时（20%的概率），处理器会从主内存中获取数据。使用公式 8.2，我们计算平均内存访问时间如下：1个周期 + 0.05[10个周期 + 0.2(100个周期)] = 2.5个周期。

L2缓存未命中率很高，因为它只接收那些在L1缓存中未命中的“困难”内存访问。如果所有访问都直接进入L2缓存，L2未命中率将约为1%。

降低未命中率

通过改变容量、块大小和/或相联度可以减少缓存未命中。减少未命中率的第一步是理解未命中的原因。未命中可分为强制未命中、容量未命中和冲突未命中。对缓存块的第一次请求称为*compulsory miss*，因为无论缓存设计如何，该块都必须从内存中读取。当缓存太小而无法容纳所有同时使用的数据时，会发生*Capacity misses*。当多个地址映射到同一组并驱逐仍然需要的块时，会导致*Conflict misses*。

更改缓存参数可能会影响一种或多种类型的缓存未命中。例如，增加缓存容量可以减少冲突未命中和容量未命中，但不会影响强制性未命中。另一方面，增加块大小可能会减少强制性未命中（由于空间局部性），但实际上可能会*increase*冲突未命中（因为更多地址会映射到同一组并可能发生冲突）。

内存系统的复杂性使得评估其性能的最佳方式是通过在改变缓存参数的同时运行基准测试。图8.17展示了SPEC2000基准测试中缺失率随缓存大小和关联度变化的曲线。该基准测试的强制缺失数量较少，如x轴附近的深色区域所示。正如预期，随着缓存大小增加，容量缺失减少。提高关联度（尤其是对于小型缓存）会减少曲线顶部所示的冲突缺失数量。将关联度提高到四路或八路以上，缺失率的降低幅度很小。

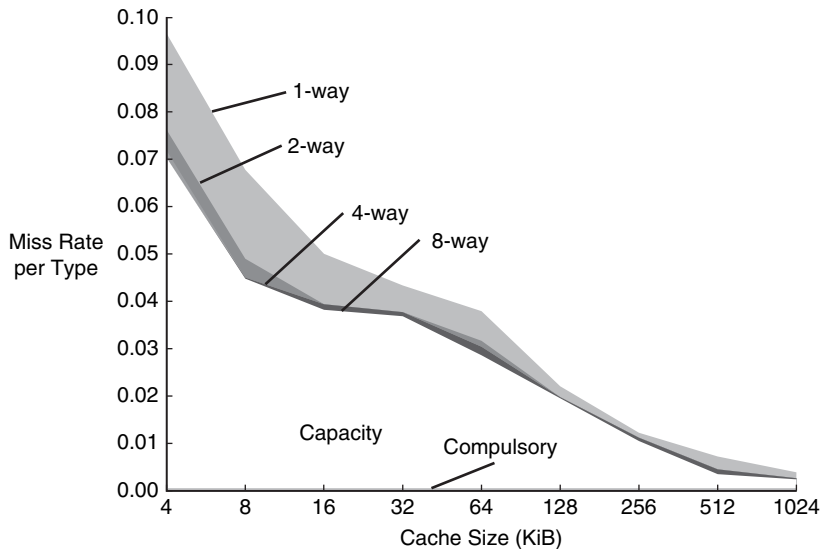


图8.17 在 SPEC2000基准测试上，缓存大小和关联性对缺失率的影响 经Hennessy与Patterson许可改编，*Computer Architecture: A*，第5版，Morgan Kaufmann，2012

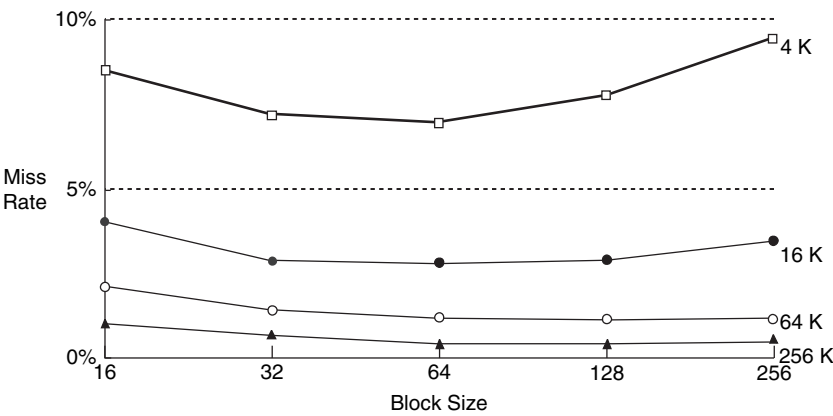


图8.18 SPEC92基准测试中缺失率与块大小和缓存大小的关系 经Hennessy和Patterson许可改编, *Computer Architecture: A Quantitative Approach*, 第5版, Morgan Kaufmann, 2012年

如前所述, 通过利用空间局部性采用更大的块大小也可以降低未命中率。但随着块大小的增加, 固定大小缓存中的组数会减少, 从而增加冲突的概率。图8.18绘制了不同容量缓存的未命中率与块大小(以字节数表示)的关系。对于小型缓存(如4 KiB缓存), 当块大小超过64字节时, 由于冲突, 未命中率会*increases*。对于较大的缓存, 将块大小增加到64字节以上并不会改变未命中率。然而, 较大的块大小仍可能因更大的未命中惩罚(即从主存获取缺失缓存块所需的时间)而增加执行时间。

写入策略

前几节重点讨论了内存加载。内存存储(即写入)遵循与加载类似的过程。当进行内存存储时, 处理器会检查缓存。如果缓存未命中, 则从主存中取出缓存块放入缓存, 然后写入缓存块中的相应字。如果缓存命中, 则直接将字写入缓存块。

缓存分为写直达和写回两种。在*write-through*缓存中, 写入缓存块的数据会同时写入主存。在*write-back*缓存中, 每个缓存块关联一个*dirty bit* (*D*)。 *D*为1表示缓存块已被写入, 为0则表示未写入。脏缓存块仅在从缓存中驱逐时才写回主存。写直达缓存不需要脏位, 但通常比写回缓存需要更多的主存写入操作。现代缓存通常采用写回方式, 因为主存访问时间过长。

示例 8.12 写直达与写回

假设一个缓存的块大小为四个字。当使用以下写入策略时，以下代码需要多少次主存访问：写直达或写回？

```
addi t5, zero, 0 sw t1
, 0(t5) sw t2, 12(t5)
sw t3, 8(t5) sw t4, 4
(t5)
```

解决方案 所有四条存储指令都写入同一个缓存块。对于写直达缓存，每条存储指令将一个字写入主存，需要四次主存写入。写回策略仅在脏缓存块被逐出时，需要一次主存访问。

8.4 虚拟内存

大多数现代计算机系统使用由磁性或固态存储构成的*hard drive*作为内存层次结构中的最底层（见图8.4）。与理想中的大容量、快速、廉价内存相比，硬盘虽然容量大且成本低，但速度极慢。它提供的存储容量远超经济高效的主存（DRAM）所能达到的水平。然而，如果内存访问中有相当一部分涉及硬盘，性能将非常糟糕。在个人电脑上同时运行过多程序时，您可能遇到过这种情况。

图8.19展示了一块由磁性存储制成的硬盘，也称为*hard disk*，其外壳盖子已被移除。顾名思义，硬盘包含一个或多个刚性盘片或*platters*，每个盘片末端的长三角臂上装有一个*read/write head*。磁头移动到磁盘上的正确位置，并在磁盘旋转时以磁性方式读取或写入数据。磁头需要几毫秒才能*seek*到磁盘上的正确位置，从人类视角来看这很快，但比处理器慢数百万倍。由于固态硬盘的读取速度要快数个数量级（见图8.4），且不易发生机械故障，机械硬盘正逐渐被固态硬盘取代。

在内存层次结构中添加硬盘的目标是，以低成本营造出拥有极大内存的假象，同时仍能为大多数访问提供更快内存的速度。例如，一台仅有16 GiB DRAM的计算机，可以通过硬盘有效提供128 GiB的内存。这个更大的128 GiB内存被称为*virtual memory*，而较小的16 GiB主内存则被称为*physical*

图8.19 硬盘



*memory*我们将使用术语*physical memory*来指代主存。在本节中。

程序可以访问虚拟内存中的任何数据，因此必须使用指定虚拟内存位置的*virtual addresses*。物理内存保存了最近访问的虚拟内存子集。通过这种方式，物理内存充当了虚拟内存的缓存。因此，大多数访问在物理内存中以DRAM的速度命中，而程序则享有更大虚拟内存的容量。

虚拟内存系统对第8.3节讨论的相同缓存原理使用了不同的术语。表8.3总结了这些对应术语。虚拟内存被划分为*virtual pages*，通常大小为4 KiB。物理内存同样被划分为相同大小的*physical pages*。一个虚拟页面可能位于物理内存（DRAM）或硬盘上。例如，图8.20展示了一个比物理内存更大的虚拟内存。矩形表示页面。部分虚拟页面存在于物理内存中，部分位于硬盘上。确定物理

一台拥有32位地址的计算机最多可以访问 2^{32} 字节=4 GiB的内存。这是转向64位计算机的动机之一，后者可以访问更多的内存。

表8.3 缓存与虚拟内存的类比术语

Cache	Virtual Memory
Block	Page
Block size	Page size
Block offset	Page offset
Miss	Page fault
Tag	Virtual page number

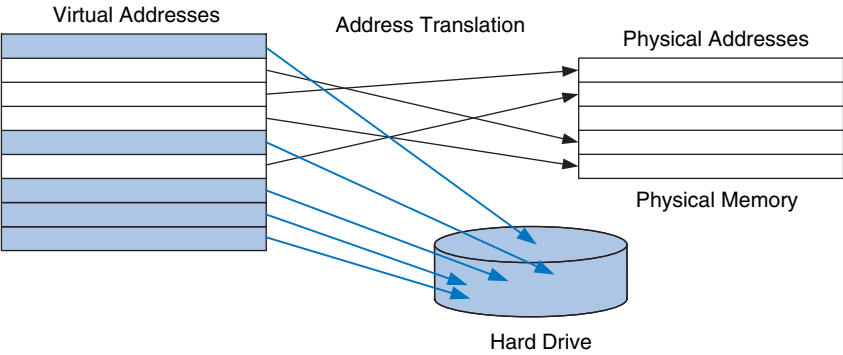


图 8.20 虚拟页面和物理页面

从虚拟地址中获取地址的过程称为`address translation`。如果处理器尝试访问一个不在物理内存中的虚拟地址，则会发生`page fault`，操作系统（OS）会将该页面从硬盘加载到物理内存中。

为避免因冲突导致的缺页，任何虚拟页面均可映射至任意物理页面。换言之，物理内存表现为虚拟内存的全相联缓存。在传统全相联缓存中，每个缓存块均配备比较器，通过将最高有效地址位与标签进行比对，以判断请求是否命中该块。在类似的虚拟内存系统中，每个物理页面需配备比较器，通过将最高有效虚拟地址位与标签进行比对，以判断虚拟页面是否映射至该物理页面。

一个真实的虚拟内存系统拥有如此多的物理页面，以至于为每个页面提供一个比较器会过于昂贵。相反，虚拟内存系统使用页表来执行地址转换。页表为每个虚拟页面包含一个条目，指示其在物理内存中的位置，或者表明它位于硬盘上。每条加载或存储指令都需要先访问页表，再进行物理内存访问。页表访问将虚拟地址进行转换。

程序使用的物理地址。然后，该物理地址用于实际读取或写入数据。

页表通常非常大，因此位于物理内存中。因此，每次加载或存储都涉及两次物理内存访问：一次页表访问和一次数据访问。为了加速地址转换，转译后备缓冲器（TLB）缓存了最常用的页表条目。

本节剩余部分将详细阐述地址转换、页表及TLB。

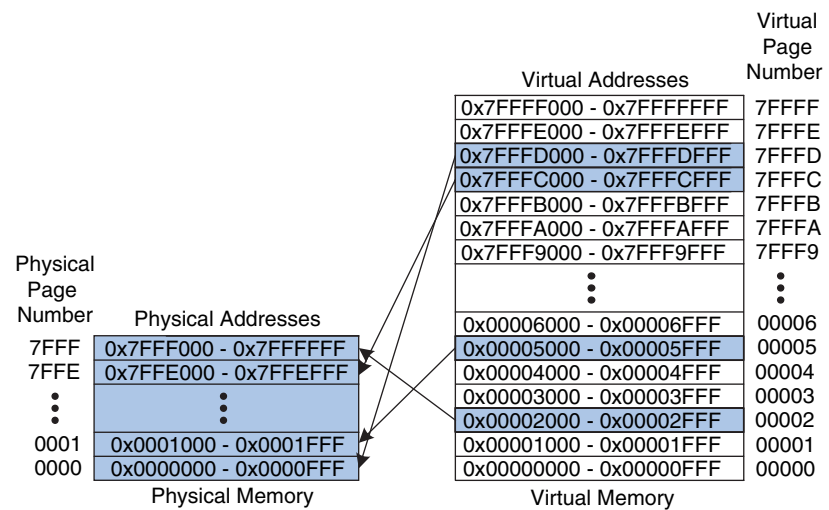
8.4.1 地址转换

在具有虚拟内存的系统中，程序使用虚拟地址以便访问大容量内存。计算机必须将这些虚拟地址转换为物理内存中的地址，或者触发缺页中断并从硬盘获取数据。

回忆一下，虚拟内存和物理内存被划分为页面。虚拟地址或物理地址的最高有效位指定虚拟或物理的*page number*。最低有效位指定页面内的字，称为*page offset*。

图8.21展示了一个虚拟内存系统的页面组织，该系统具有2 GiB虚拟内存和128 MiB物理内存，并划分为4 KiB页面。MIPS支持32位地址。对于2 GiB = 2³¹字节的虚拟内存，仅使用最低31位虚拟地址位；第32位始终为0。类似地，对于128 MiB = 2²⁷字节的物理内存，仅使用最低27位物理地址位；高5位始终为0。

图 8.21 物理页和虚拟页



由于页面大小为4 KiB = 2¹²字节，因此有2³¹/2¹² = 2¹⁹个虚拟页面和2²⁷/2¹² = 2¹⁵个物理页面。因此，虚拟和物理页号分别为19位和15位。物理内存存在任何时候最多只能容纳虚拟页面的1/16。其余虚拟页面保存在硬盘上。

图8.21展示了虚拟页5映射到物理页1，虚拟页0x7FFFC映射到物理页0x7FFE，以此类推。例如，虚拟地址0x53F8（虚拟页5内偏移量0x3F8）映射到物理地址0x13F8（物理页1内偏移量0x3F8）。虚拟地址和物理地址的最低12位相同（0x3F8），指定了虚拟页和物理页内的页偏移量。只需转换页号即可从虚拟地址得到物理地址。

图8.22展示了虚拟地址到物理地址的转换过程。最低的12位表示页内偏移，无需转换。虚拟地址的高19位指定了*virtual page number*（VPN），并被转换为一个15位的*physical page number*（PPN）。接下来的两节将描述如何使用页表和TLB来执行这一地址转换。

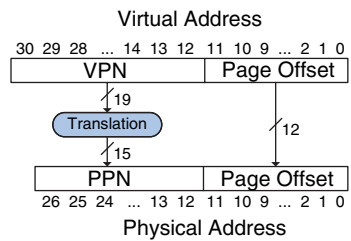


图8.22 从虚拟地址到物理地址的转换

示例8.13 虚拟地址到物理地址的转换

使用图8.21所示的虚拟内存系统，查找虚拟地址0x247C的物理地址。

解决方案 12位页偏移量（0x47C）无需转换。虚拟地址剩余的19位给出虚拟页号，因此虚拟地址0x247C位于虚拟页0x2中。在图8.21中，虚拟页0x2映射到物理页0x7FFF。因此，虚拟地址0x247C映射到物理地址0x7FFF47C。

8.4.2 页表

处理器使用*page table*将虚拟地址转换为物理地址。页表为每个虚拟页包含一个条目。该条目包含物理页号和有效位。如果有效位

如果为1，则虚拟页映射到条目中指定的物理页。否则，虚拟页位于硬盘上。

由于页表非常大，因此它存储在物理内存中。我们暂时假设它像图8.23所示那样以连续数组的形式存储。该页表包含了图8.21中内存系统的映射关系。页表通过虚拟页号（VPN）进行索引。例如，条目5指定虚拟页5映射到物理页1。条目6无效（ $V = 0$ ），因此虚拟页6位于硬盘上。

V	Physical Page Number	Virtual Page Number
0		7FFFF
0		7FFFE
1	0x0000	7FFFD
1	0x7FFE	7FFFC
0		7FFFB
0		7FFFA
	⋮	⋮
0		00007
0		00006
0		00005
1	0x0001	00004
0		00003
0		00002
1	0x7FFF	00001
0		00000

Page Table

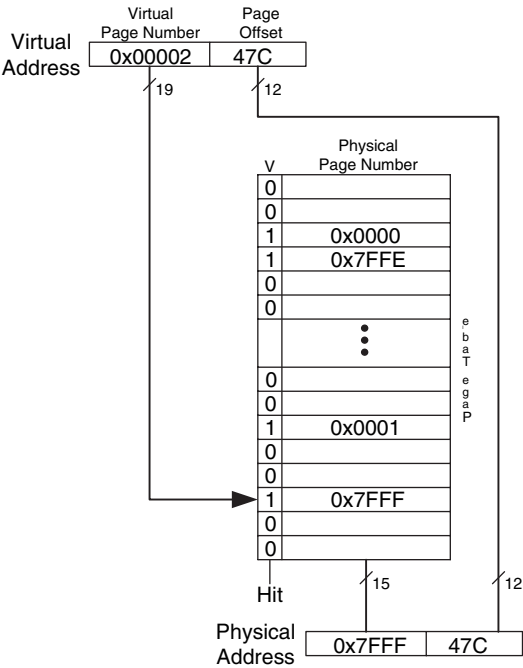
图8.23 图8.21的页表

示例8.14 使用页表执行地址转换

使用图8.23所示的页表查找虚拟地址0x247C的物理地址。

解 图8.24展示了虚拟地址0x247C到物理地址的转换。12位页内偏移无需转换。虚拟地址剩余的19位是虚拟页号0x2，用于索引页表。页表将虚拟页0x2映射到物理页0x7FFF。因此，虚拟地址0x247C映射到物理地址0x7FFF47C。物理地址与虚拟地址的最低12位相同。

图8.24 使用页表进行地址转换



页表可以存储在物理内存中的任何位置，由操作系统决定。处理器通常使用一个专用寄存器，称为`page table register`，来存储页表在物理内存中的基地址。

要执行加载或存储操作，处理器必须首先将虚拟地址转换为物理地址，然后访问该物理地址处的数据。处理器从虚拟地址中提取虚拟页号，并将其添加到页表寄存器中，以找到页表项的物理地址。接着，处理器从物理内存中读取该页表项，以获取物理页号。如果该项有效，则将该物理页号与页内偏移量合并，生成物理地址。最后，处理器在该物理地址处读取或写入数据。由于页表存储在物理内存中，每次加载或存储操作都涉及两次物理内存访问。

8.4.3 转译后备缓冲器

如果每次加载或存储都需要读取页表，虚拟内存将导致严重的性能影响，使加载和存储的延迟加倍。幸运的是，页表访问具有很强的时间局部性。数据访问的时间和空间局部性以及较大的页面尺寸意味着许多连续的加载或存储很可能引用同一页面。因此，如果处理器记住它上次读取的页表项，它很可能可以重用这个转换，而无需重新读取页表。通常，处理器可以将最近几个页表项保存在一个称为`translation lookaside buffer` (TLB) 的小型缓存中。处理器在访问物理内存中的页表之前，会先“旁路”查找TLB中的转换。在实际程序中，绝大多数访问都能在TLB中命中，从而避免了从物理内存读取页表这一耗时操作。

TLB 被组织为全相联缓存，通常包含 16 到 512 个条目。每个 TLB 条目保存一个虚拟页号及其对应的物理页号。TLB 通过虚拟页号进行访问。如果 TLB 命中，则返回对应的物理页号；否则，处理器必须读取物理内存中的页表。TLB 设计得足够小，以便在一个周期内完成访问。即便如此，TLB 的命中率通常仍高于 99%。TLB 将大多数加载或存储指令所需的内存访问次数从两次减少到一次。

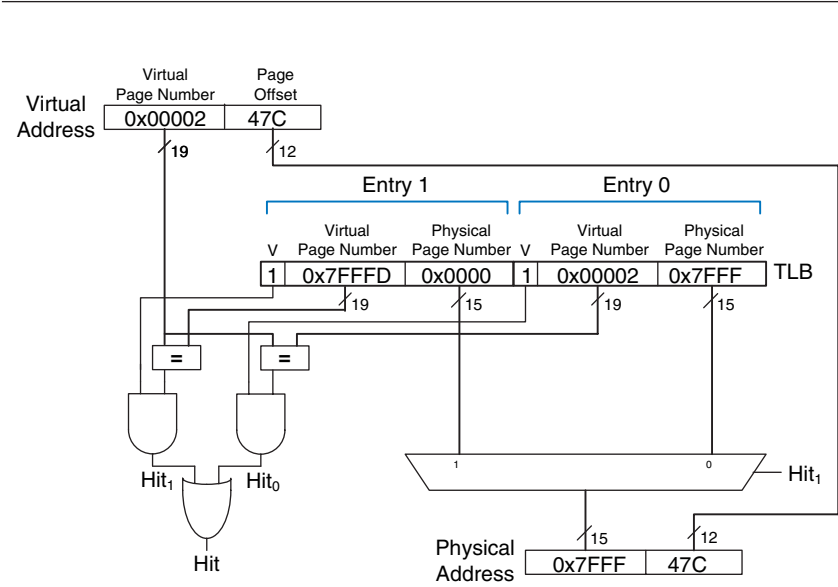
示例8.15 使用TLB执行地址转换

考虑图8.21的虚拟内存系统。使用一个两条目TLB，或解释为何需要访问页表来将虚拟地址0x247C和0x5FB0转换为物理地址。假设TLB当前持有虚拟页0x2和0x7FFFD的有效转换。

解法 图8.25展示了双条目TLB接收到虚拟地址0x247C的请求。TLB获取传入地址的虚拟页号0x2，并将其与每个条目的虚拟页号进行比较。条目0匹配且有效，因此请求命中。转换后的物理地址是匹配条目的物理页号0x7FFF与虚拟地址的页内偏移拼接而成。与往常一样，页内偏移无需转换。

对虚拟地址0x5FB0的请求在TLB中未命中。因此，该请求被转发到页表进行转换。

图 8.25 使用双条目 TLB 进行地址转换



8.4.4 内存保护

到目前为止，本节主要关注使用虚拟内存来提供快速、廉价且容量大的内存。使用虚拟内存的另一个同样重要的原因是为并发运行的程序之间提供保护。

你可能知道，现代计算机通常同时运行多个程序或*processes*。所有程序同时存在于物理内存中。在精心设计的计算机系统中，程序之间应相互保护，以确保没有程序能够崩溃或劫持另一个程序。具体来说，未经许可，任何程序都不应能够访问另一个程序的内存。这被称为*memory protection*。

虚拟内存系统通过为每个程序提供其自己的*virtual address space*来实现内存保护。每个程序可以在该虚拟地址空间中使用任意多的内存，但只有一部分

虚拟地址空间在任何给定时刻都位于物理内存中。每个程序都可以使用其完整的虚拟地址空间，而无需担心其他程序在物理上的位置。然而，程序只能访问其页表中映射的那些物理页面。通过这种方式，程序无法意外或恶意地访问其他程序的物理页面，因为这些页面并未映射到其页表中。在某些情况下，多个程序会访问共同的指令或数据。操作系统会向每个页表条目添加控制位，以确定哪些程序（如果有的话）可以写入共享的物理页面。

8.4.5 替换策略*

虚拟内存系统采用写回策略和近似最近最少使用（LRU）替换策略。写直达策略（每次对物理内存的写入都会触发对硬盘的写入）是不切实际的。存储指令将以硬盘的速度而非处理器的速度运行（毫秒级而非纳秒级）。在写回策略下，物理页仅在从物理内存中被驱逐时才写回硬盘。将物理页写回硬盘并重新加载另一个虚拟页的过程称为*paging*，虚拟内存系统中的硬盘有时被称为*swap space*。当发生缺页时，处理器会换出最近最少使用的物理页，然后用缺失的虚拟页替换该页。为支持这些替换策略，每个页表项包含两个额外的状态位：脏位*D*和使用位*U*。

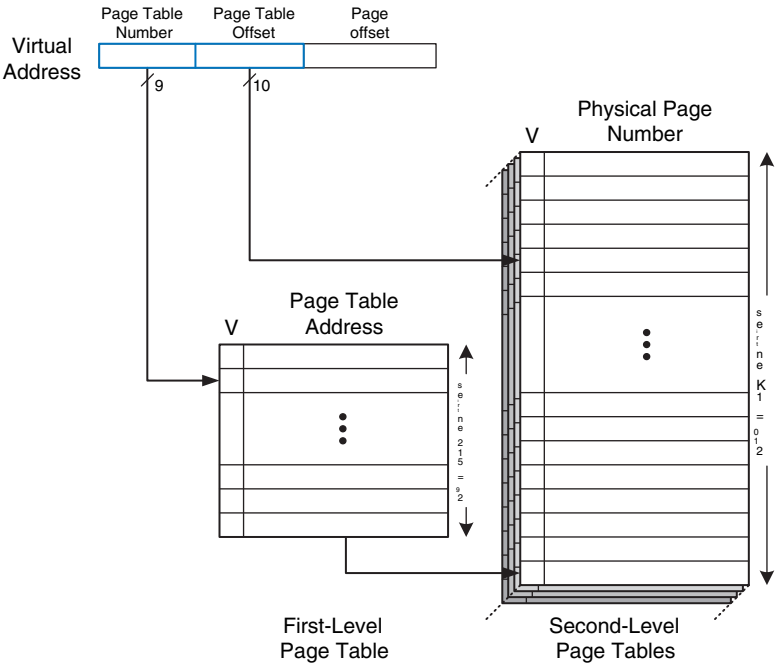
脏位为1表示自物理页从硬盘读取后，有任何存储指令修改了该页。当物理页被换出时，仅当其脏位为1时才需要写回硬盘；否则，硬盘中已存有该页的精确副本。

如果物理页面最近被访问过，则使用位为1。与缓存系统类似，精确的LRU替换会过于复杂而难以实现。因此，操作系统通过定期重置页表中所有使用位来近似实现LRU替换。当页面被访问时，其使用位被设置为1。发生缺页时，操作系统会找到一个*U* = 0的页面将其换出物理内存。因此，它不一定替换最近最少使用的页面，而只是替换其中一个最近最少使用的页面。

8.4.6 多级页表*

页表会占用大量物理内存。例如，上一节中针对2 GiB虚拟内存、4 KiB页面的页表需要 2^{19} 个条目。如果每个条目为4字节，则该页表大小为 $2^{19} \times 2^2 \text{ 字节} = 2^{21} \text{ 字节} = 2 \text{ MiB}$ 。

图8.26 分层页表



为节省物理内存，页表可拆分为多个（通常为两级）层级。第一级页表始终驻留于物理内存中，用于指示较小的第二级页表在虚拟内存中的存储位置。每个第二级页表包含对应虚拟页面范围的真实地址转换。若某段地址转换范围未被频繁使用，其对应的第二级页表可被换出至硬盘，从而避免浪费物理内存。

在两级页表中，虚拟页号被分为两部分：*page table number*和*page table offset*，如图8.26所示。页表编号索引第一级页表，该页表必须驻留在物理内存中。第一级页表项给出第二级页表的基地址，或者当V为0时指示必须从硬盘中获取该页表。页表偏移量索引第二级页表。虚拟地址中剩余的12位是页内偏移量，与之前相同，对应页面大小为 $2^{12} = 4$ KiB。

在图8.26中，19位虚拟页号被拆分为9位和10位，分别用于指示页表编号和页表偏移量。因此，第一级页表有 $2^9 = 512$ 个条目。这512个第二级页表中的每一个都有 $2^{10} = 1$ Ki个条目。如果第一级和第二级页表条目均为32位（4字节），且物理内存中同时仅存在两个第二级页表，则

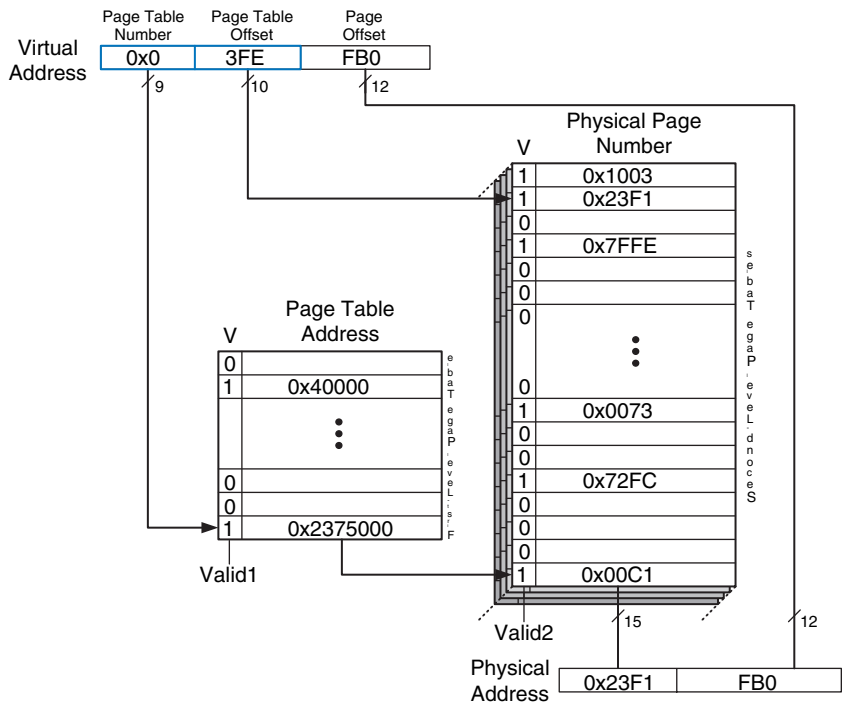


图8.27 使用两级页表的地址转换

分层页表仅使用 $(512 \times 4 \text{ 字节}) + 2 \times (1 \text{ Ki} \times 4 \text{ 字节}) = 10 \text{ KiB}$ 的物理内存。两级页表所需的物理内存仅为存储整个页表(2 MiB)所需物理内存的一小部分。两级页表的缺点是，当TLB未命中时，它会为地址转换增加一次额外的内存访问。

示例8.16 使用多级页表进行地址转换

图8.27展示了图8.26中两级页表的可能内容。仅显示了一个二级页表的内容。使用此两级页表，描述访问虚拟地址0x003FEFB0时会发生什么。

解决方案 与往常一样，只有虚拟页号需要转换。虚拟地址的最高九位(0x0)给出页表号，即一级页表的索引。一级页表中索引0x0处的条目表明二级页表驻留在内存中(V=1)，其物理地址为0x2375000。

虚拟地址接下来的十位，0x3FE，是页表偏移量，用于给出二级页表的索引。条目0位于二级页表的底部，条目0x3FF位于顶部。二级页表中的条目0x3FE表示该虚拟页驻留在物理内存中（ $V = 1$ ），且物理页号为0x23F1。物理页号与页内偏移量拼接形成物理地址0x23F1FB0。

8.5 总结

存储器系统组织是决定计算机性能的主要因素。不同的存储器技术（如DRAM、SRAM和硬盘）在容量、速度和成本之间各有取舍。本章介绍了缓存和虚拟存储器组织，它们利用存储器层次结构来近似实现理想的大容量、高速、低成本存储器。主存储器通常由DRAM构成，其速度明显慢于处理器。缓存通过将常用数据保存在高速SRAM中来减少访问时间。虚拟存储器则利用硬盘存储主存中放不下的数据，从而增加存储器容量。缓存和虚拟存储器增加了计算机系统的复杂性和硬件开销，但其收益通常超过成本。所有现代个人计算机都使用缓存和虚拟存储器。大多数处理器还通过存储器接口与输入/输出（I/O）设备进行通信，这被称为 *memory-mapped I/O*。程序使用加载和存储操作来访问I/O设备，相关内容将在第9章（本书配套网站上提供的在线补充章节，参见前言）中讨论。

后记

本章标志着我们共同探索数字系统世界的旅程的终点。我们希望本书不仅传递了工程知识，也展现了这门艺术的美妙与激情。你已学会使用原理图和硬件描述语言设计组合逻辑与时序逻辑，熟悉了多路选择器、算术逻辑单元和存储器等大型构建模块。计算机是数字系统最迷人的应用之一。你掌握了用RISC-V原生汇编语言编程的方法，以及如何利用数字构建模块搭建处理器与存储系统。贯穿全书，你见证了抽象、规范、层次化、模块化和规律性的应用。借助这些技术，我们逐步拼凑出微处理器内部运作的拼图。从手机到数字电视，从火星探测器到医学成像系统，我们的世界正日益数字化。

想象一下，一个半世纪前，查尔斯·巴贝奇会做出怎样的浮士德式交易，才能踏上类似的旅程。他仅仅渴望用机械精度计算数学表格。如今的数字系统是昨日的科幻小说。迪克·特雷西会在他的手机上听iTunes吗？儒勒·凡尔纳会将全球定位卫星星座发射到太空吗？希波克拉底能用高分辨率的大脑数字图像治愈疾病吗？但与此同时，乔治·奥威尔笔下无处不在的政府监控噩梦正日益成为现实。黑客和政府发动未宣战的网络战争，攻击工业基础设施和金融网络。而流氓国家使用比冷战时期模拟核弹的房间大小超级计算机更强大的笔记本电脑来开发核武器。微处理器革命持续加速。未来几十年的变化将超越过去。你现在拥有了设计和构建这些将塑造未来的新系统的工具。伴随你新获得的力量而来的是深重的责任。我们希望你运用它，不仅为了乐趣和财富，也为了人类的福祉。

练习

练习8.1 在不到一页的篇幅内，描述四种体现时间或空间局部性的日常活动。针对每种局部性列出两项活动，并具体说明。

练习8.2 用一段话描述两个展示时间局部性和/或空间局部性的简短计算机应用程序。具体说明它们是如何体现的。

练习8.3 提出一个地址序列，使得一个容量为16字、块大小为4字的直接映射缓存的性能优于一个具有相同容量和块大小、采用最近最少使用（LRU）替换策略的全相联缓存。

练习8.4 针对全相联缓存性能优于直接映射缓存的情况，重复练习8.3。

练习8.5 描述在保持其他参数不变的情况下，增加以下每个缓存参数所带来的权衡：

- (a) 块大小
- (b) 结合律
- (c) 缓存大小

练习8.6 一个两路组相联缓存的缺失率是否总是、通常、偶尔或从不优于相同容量和块大小的直接映射缓存？请解释。

练习8.7 以下每个陈述都与缓存的未命中率有关。将每个陈述标记为正确或错误。简要解释你的推理；如果陈述错误，请给出一个反例。

- (a) 在相同块大小和总容量的情况下，两路组相联缓存的缺失率总是低于直接映射缓存。
- (b) 在相同块大小的情况下，16 KiB直接映射缓存的缺失率总是低于8 KiB直接映射缓存。
- (c) 在相同相联度和总容量的情况下，块大小为32字节的指令缓存通常比块大小为8字节的指令缓存具有更低的缺失率。

练习8.8 一个缓存具有以下参数： b ，以字数为单位的块大小； S ，组数； N ，路数；以及 A ，地址位数。

- (a) 根据所描述的参数，缓存容量 C 是多少？ (b) 根据所描述的参数，存储标签所需的总位数是多少？ (c) 对于容量为 C 字、块大小为 b 的全相联缓存， S 和 N 是多少？ (d) 对于大小为 C 字、块大小为 b 的直接映射缓存， S 是多少？

练习8.9 一个16字缓存具有练习8.8中给出的参数。考虑以下重复的1w地址序列（以十六进制给出）：

40 44 48 4 70 74 78 7 80 84 88 8 90 94 98 9 0 48 10 14 18 1 20 C C C C C C

假设关联缓存采用最近最少使用（LRU）替换策略，忽略启动效应（即强制缺失），确定以下序列输入到缓存时的有效缺失率。

- (a) 直接映射缓存， $b = 1$ 字 (b) 全相联缓存， $b = 1$ 字 (c) 两路组相联缓存， $b = 1$ 字 (d) 直接映射缓存， $b = 2$ 字

练习8.10 对于以下1w地址（以十六进制给出）的重复序列和缓存配置，重复练习8.9。缓存容量仍为16个字。

74 A0 78 38C AC 84 88 8C 7C 34 38 13C 388 18C

- (a) 直接映射缓存， $b = 1$ 字 (b) 全相联缓存， $b = 2$ 字 (c) 两路组相联缓存， $b = 2$ 字 (d) 直接映射缓存， $b = 4$ 字

练习8.11 假设你正在运行一个程序，其数据访问模式如下（以十六进制给出）。该模式仅执行一次。

0 8 10 18 20 28

(a) 如果使用直接映射缓存，缓存大小为 1 KiB，块大小为 8 字节（2 个字），缓存中有多少个组？ (b) 使用与 (a) 部分相同的缓存和块大小，对于给定的内存访问模式，直接映射缓存的未命中率是多少？

(c) 对于给定的内存访问模式，以下哪项最能降低未命中率？（缓存容量保持不变。）请圈出一项。

(i) 将关联度提高到2。

(ii) 将块大小增加到16字节。

(iii) 要么 (i) 要么 (ii)。

(iv) 既非(i)亦非(ii)。

练习8.12 你正在为RISC-V处理器构建一个指令缓存。其总容量为 $4C = 2^{c+2}$ 字节。它是 $N = 2^n$ 路组相联（ $N \geq 8$ ），块大小为 $b = 2^{b'}$ 字节（ $b \geq 8$ ）。请根据这些参数回答以下问题。

(a) 地址中的哪些位用于在块内选择一个字 ck?

(b) 地址中的哪些位用于选择缓存中的组？

(c) 每个标签有多少位？ (d) 整个缓存中有多少标签位？

练习8.13 考虑一个具有以下参数的缓存：

N (结合性) = 2, b (块大小) = 2 字, W (字大小) = 32 位, C (缓存大小) = 32 K

i 字, A (地址大小) = 32 位。你需要考虑

仅字地址。

(a) 显示地址的标记、集合、块偏移和字节偏移位。说明每个字段需要多少位。

(b) *all* 缓存标签的大小是多少（以位为单位）？

(c) 假设每个缓存块还有一个有效位(V)和一个脏位(D)。每个缓存集的大小是多少，包括数据、标签和状态位？ (d) 使用图8.28中的构建块和少量二输入逻辑门设计该缓存。缓存设计必须包括标签存储、数据

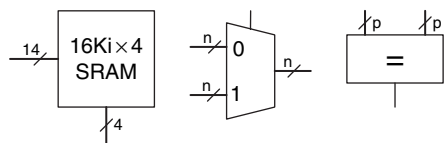


图8.28 构建块

存储、地址比较、数据输出选择，以及您认为相关的任何其他部分。请注意，多路复用器和比较器模块可以是任意大小（分别为 n 位或 p 位宽），但SRAM模块必须是 $16\text{Ki} \times 4$ 位。请务必包含一个标注清晰的模块图。您只需设计用于读取的缓存。

练习8.14 你加入了一家热门的新互联网初创公司，致力于制造内置寻呼机和网页浏览器的手表。该手表采用嵌入式处理器，并配有如图8.29所示的多级缓存方案。处理器除了一个大型片外二级缓存外，还包含一个小型片内缓存。（没错，这款手表重达3磅，但你应该看看它上网冲浪的样子！）

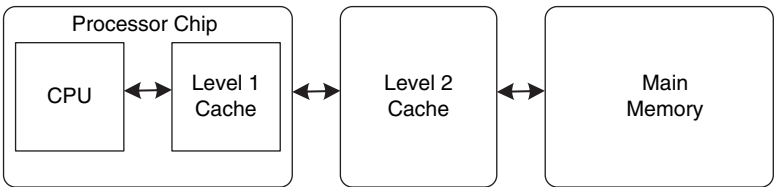


图8.29 计算机系统

假设处理器使用32位物理地址，但仅在字边界上访问数据。缓存具有表8.4中给出的特性。DRAM的访问时间为 t_m ，大小为512 MiB。

表8.4 内存特性

Characteristic	On-chip Cache	Off-chip Cache
Organization	Four-way set associative	Direct mapped
Hit rate	A	B
Access time	t_a	t_b
Block size	16 bytes	16 bytes
Number of blocks	512	256 Ki

(a) 对于内存中的给定字，它在片上缓存和二级缓存中可能被找到的位置总数是多少？(b) 片上缓存和二级缓存的每个标签的大小（以位为单位）是多少？(c) 给出平均内存读取访问时间的表达式。缓存按顺序访问。(d) 测量显示，对于某个特定问题，片上缓存命中率为85%，二级缓存命中率为90%。然而，当片上缓存被禁用时，二级缓存命中率飙升至98.5%。请简要解释这一行为。

练习8.15 本章描述了多路组相联缓存中最近最少使用（LRU）替换策略。其他不太常见的替换策略包括先进先出（FIFO）和随机策略。FIFO替换会驱逐在缓存中停留时间最长的块，无论其最近是否被访问过。随机替换则随机选择一个块进行驱逐。

(a) 讨论每种替换策略的优缺点。

(b) 描述一种数据访问模式，使得 FIFO 比 LRU 表现更好。

练习8.16 你正在构建一台具有分层存储系统的计算机，该系统由独立的指令缓存和数据缓存以及主存组成。你使用的是图7.44中的RISC-V多周期处理器，运行频率为1 GHz。

(a) 假设指令缓存是完美的（即始终命中），但数据缓存的缺失率为5%。在缓存缺失时，处理器会停顿60纳秒以访问主存，然后恢复正常操作。考虑缓存缺失的情况，平均内存访问时间是多少？

(b) 考虑到非理想内存系统，加载和存储字指令平均每个指令需要多少个时钟周期（CPI）？

(c) 考虑示例7.7中的基准应用程序，其中包含25%的加载指令、10%的存储指令、11%的分支指令、2%的跳转指令和52%的R型指令³。考虑到非理想内存系统，该基准的平均CPI是多少？

(d) 现在假设指令缓存也不是理想的，且缺失率为7%。对于部分(c)中的基准测试，平均CPI是多少？请同时考虑指令缓存和数据缓存的缺失。

³ Data from Patterson and Hennessy, *Computer Organization and Design*, 4th Edition, Morgan Kaufmann, 2011. Used with permission.

练习8.17 使用以下参数重复练习8.16。

- (a) 指令缓存完美（即始终命中），但数据缓存有15%的缺失率。发生缓存缺失时，处理器停顿200纳秒以访问主存，然后恢复正常操作。考虑缓存缺失，平均内存访问时间是多少？
- (b) 考虑到非理想内存系统，加载和存储字指令平均每个指令需要多少个时钟周期（CPI）？
- (c) 考虑示例7.7中的基准应用程序，其中包含25%的加载指令、10%的存储指令、11%的分支指令、2%的跳转指令和52%的R型指令。考虑到非理想内存系统，该基准的平均CPI是多少？
- (d) 现在，假设指令缓存也不理想，且缺失率为10%。对于部分(c)中的基准测试，平均CPI是多少？请同时考虑指令缓存和数据缓存的缺失。

练习8.18 如果一台计算机使用64位虚拟地址，它可以访问多少虚拟内存？注意， 2^{40} 字节 = 1 *terabyte (tebibyte)*， 2^{50} 字节 = 1 *petabyte (pebibyte)*，以及 2^{60} 字节 = 1 *exabyte (exbibyte)*。

练习8.19 一位超级计算机设计师选择花费100万美元购买DRAM，并以相同金额购买用于虚拟内存的硬盘。根据图8.4中的价格，该计算机将拥有多少物理内存和虚拟内存？访问这些内存需要多少位的物理地址和虚拟地址？

练习8.20 考虑一个虚拟内存系统，其可寻址总空间为 2^{32} 字节。你拥有无限的硬盘空间，但仅限使用8 MiB的半导体（物理）内存。假设虚拟页面和物理页面的大小均为4 KiB。

- (a) 物理地址有多少位？ (b) 系统中虚拟页的最大数量是多少？ (c) 系统中有多少物理页？ (d) 虚拟页号和物理页号各有多少位？ (e) 假设你提出一种直接映射方案，将虚拟页映射到物理页。该映射使用虚拟页号的最低有效位来确定物理页号。每个物理页映射了多少个虚拟页？为什么这种“直接映射”是一个糟糕的方案？

(f) 显然，需要一种比(e)部分所述方案更灵活、更动态的虚拟地址到物理地址转换机制。假设你使用页表来存储映射关系（从虚拟页号到物理页号的转换）。那么该页表将包含多少个页表项？

(g) 假设除了物理页号之外，每个页表项还包含一些状态信息，形式为一个有效位（V）和一个脏位（D）。每个页表项的长度是多少字节？（向上取整为整数字节。）

(h) 画出页表的布局。页表的总大小是多少字节？

练习8.21 考虑一个虚拟内存系统，其可寻址的总空间为 2^{50} 字节。你拥有无限的硬盘空间，但半导体（物理）内存限制为2 GiB。假设虚拟页面和物理页面的大小均为4 KiB。

(a) 物理地址有多少位？

(b) 系统中虚拟页面的最大数量是多少？

(c) 系统中有多少个物理页面？

(d) 虚拟页号和物理页号各有多少位？

(e) 页表将包含多少页表项？

(f) 假设除了物理页号之外，每个页表项还包含一些状态信息，形式为一个有效位（V）和一个脏位（D）。每个页表项的长度是多少字节？（向上取整为整数字节。） (g) 画出页表的布局。页表的总大小是多少字节？

练习8.22 你决定通过使用转译后备缓冲器（TLB）来加速练习8.20中的虚拟内存系统。假设你的内存系统具有表8.5所示的特性。TLB和缓存未命中率表明如何

表8.5 内存特性

Memory Unit	Access Time (Cycles)	Miss Rate
TLB	1	0.05%
Cache	1	2%
Main memory	100	0.0003%
Hard drive	1,000,000	0%

通常请求的条目未找到。主存未命中率指示页面错误发生的频率。

(a) 在添加TLB前后，虚拟内存系统的平均内存访问时间分别是多少？假设页表始终驻留在物理内存中，且从不保存在数据缓存中。(b) 如果TLB有64个条目，TLB的大小（以位为单位）是多少？请给出每个条目的数据（物理页号）、标签（虚拟页号）和有效位的具体数值。清晰展示你的计算过程。

(c) 绘制TLB草图。清晰标注所有字段和维度。

(d) 构建第(c)部分所述的TLB需要多大容量的SRAM？请以深度×宽度的形式给出答案。

练习8.23 你决定通过使用一个包含128条目的转译后备缓冲器（TLB）来加速练习8.21中的虚拟内存系统。

(a) TLB有多大（以比特计）？请给出每个表项的数据（物理页号）、标签（虚拟页号）和有效位的数值。清晰展示你的计算过程。

(b) 绘制TLB草图。清晰标注所有字段和尺寸。

(c) 构建(b)部分所述的TLB需要多大容量的SRAM？请以深度×宽度的形式给出答案。

练习8.24 假设第7.4节中描述的RISC-V多周期处理器使用虚拟内存系统。

(a) 在多周期处理器框图中标出TLB的位置。

(b) 描述添加TLB如何影响处理器性能。

练习8.25 你正在设计的虚拟内存系统使用由专用硬件（SRAM及相关逻辑）构建的单级页表。该系统支持25位虚拟地址、22位物理地址以及 2^{16} 字节（64 KiB）的页面。每个页表项包含一个物理页号、一个有效位（V）和一个脏位（D）。

(a) 页表的总大小是多少（以位为单位）？(b) 操作系统团队提议将页面大小从64减少到16 KiB，但硬件工程师团队以增加硬件成本为由提出反对。请解释他们的反对理由。

(c) 页表将与片上缓存一起集成在处理器芯片上。片上缓存仅处理物理地址（而非虚拟地址）。对于给定的内存访问，是否可以在访问页表的同时访问片上缓存的相应集合？简要说明并发访问缓存集合和页表项所需的关系。

(d) 是否可以在片内缓存中执行标签比较，同时进行给定内存访问的页表访问？简要说明。

练习8.26 描述一个虚拟内存系统可能影响应用程序编写方式的场景。请务必讨论页面大小和物理内存大小如何影响应用程序的性能。

练习8.27 假设你拥有一台使用32位虚拟地址的个人计算机（PC）。

(a) 每个程序的最大虚拟内存空间是多少
可以使用吗？(b) 电脑硬盘的大小如何影响性能？(c) 电脑物理内存的大小如何影响性能？

面试问题

以下练习题呈现了面试中曾提出的问题。

问题8.1 解释直接映射、组相联和全相联缓存之间的区别。针对每种缓存类型，描述一种该缓存类型性能优于其他两种的应用场景。

问题8.2 解释虚拟内存系统的工作原理。

问题8.3 解释使用虚拟内存系统的优缺点。

问题8.4 解释缓存性能如何受内存系统虚拟页面大小的影响。